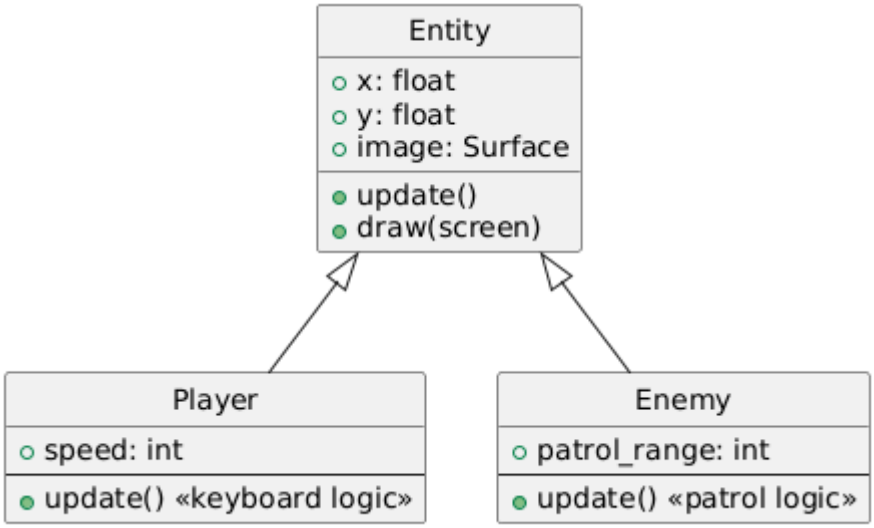
Exercise 2: Entity-Component System Design

1. UML Diagram of Inheritance Hierarchy



Limitations of the Inheritance Approach:

1. **Code Duplication:** Shared logic (e.g., movement, collision) is often repeated in multiple subclasses.
2. **Inflexibility:** Adding new behaviors (like patrolling or flying) requires modifying or creating new subclasses, making the hierarchy rigid.
3. **Difficult to Combine Behaviors:** If an entity needs multiple behaviors (e.g., a collectible that moves), you must create new subclasses for every combination, leading to class explosion.

---

2. Component-Based Architecture Proposal

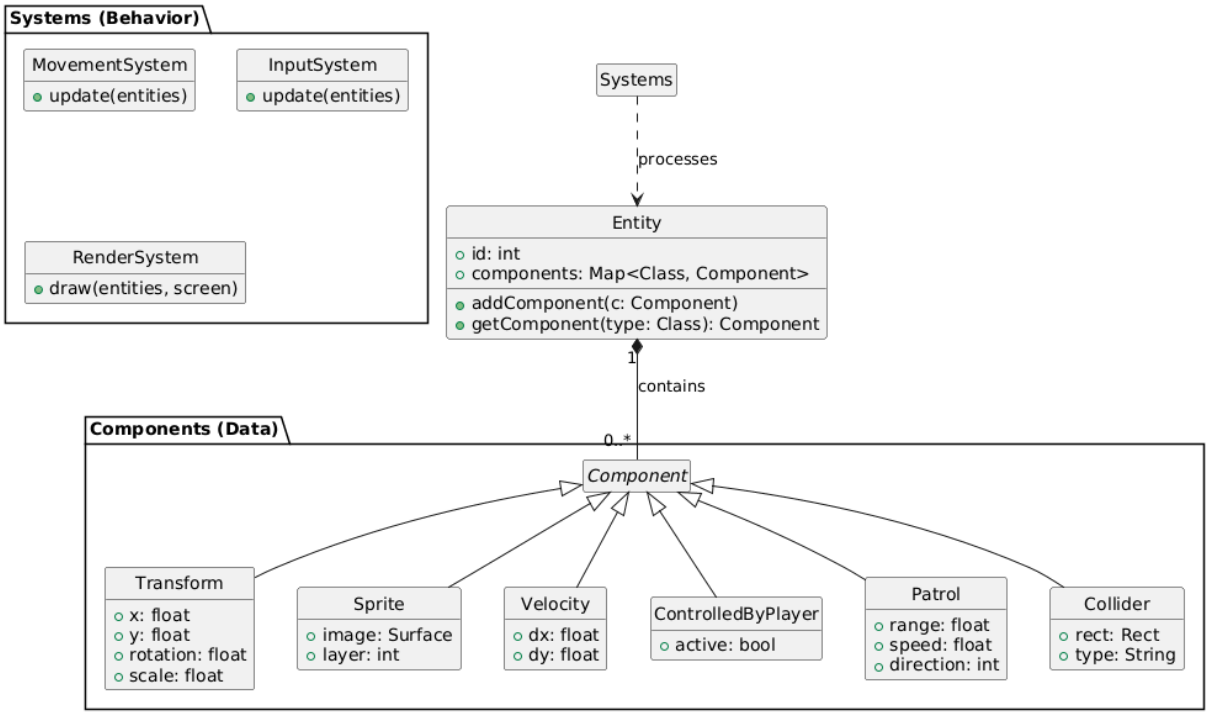
We redesign the system using an Entity-Component-System (ECS) approach:

- **Entity:** Holds a dictionary of components.
- **Component:** Simple data containers (e.g., Transform, Sprite, Velocity, etc.).
- **System:** Processes all entities with required components.

Components:

- **Transform:** position, rotation, scale
- **Sprite:** image, layer

- **Velocity:** dx, dy
- **ControlledByPlayer:** flag
- **Patrol:** range, speed, direction
- **Collider:** rect, type



---

3. Core ECS Implementation in Pygame

```
# --- Components as simple dictionaries ---
def Transform(x, y, rotation=0, scale=1):
    return {'x': x, 'y': y, 'rotation': rotation, 'scale': scale}

def Sprite(image, layer=0):
    return {'image': image, 'layer': layer}

def Velocity(dx=0, dy=0):
```

```
    return {'dx': dx, 'dy': dy}

def ControlledByPlayer():
    return {'player_controlled': True}

def Patrol(range_, speed, direction=1):
    return {'range': range_, 'speed': speed, 'direction': direction, 'start_x': None}

def Collider(rect, type_):
    return {'rect': rect, 'type': type_}

# --- Entity class ---
class Entity:
    def __init__(self):
        self.components = {}

    def add_component(self, name, component):
        self.components[name] = component

    def get_component(self, name):
        return self.components.get(name)

# --- Systems ---
class MovementSystem:
    def update(self, entities, dt):
        for entity in entities:
            transform = entity.get_component('Transform')
```

```
            velocity = entity.get_component('Velocity')
            if transform and velocity:
                transform['x'] += velocity['dx'] * dt
                transform['y'] += velocity['dy'] * dt

class RenderSystem:
    def draw(self, entities, screen):
        for entity in entities:
            transform = entity.get_component('Transform')
            sprite = entity.get_component('Sprite')
            if transform and sprite:
                x, y = int(transform['x']), int(transform['y'])
                screen.blit(sprite['image'], (x, y))
```

---

4. Example: Creating a Player and Patrolling Enemy

```
# Assume 'player_img' and 'enemy_img' are loaded pygame.Surface objects

# --- Create player entity ---
player = Entity()
player.add_component('Transform', Transform(100, 300))
player.add_component('Sprite', Sprite(player_img))
player.add_component('Velocity', Velocity())
player.add_component('ControlledByPlayer', ControlledByPlayer())
player.add_component('Collider', Collider(pygame.Rect(100, 300, 32, 32), 'player'))
```

```
# --- Create patrolling enemy entity ---
enemy = Entity()
enemy.add_component('Transform', Transform(400, 300))
enemy.add_component('Sprite', Sprite(enemy_img))
enemy.add_component('Velocity', Velocity(50, 0))  # Moves horizontally
enemy.add_component('Patrol', Patrol(range_=100, speed=50))
enemy.add_component('Collider', Collider(pygame.Rect(400, 300, 32, 32), 'enemy'))
```

---

5. Explanation: Extending with New Behaviors

With this ECS design, adding new behaviors is easy:

- To add a new feature (e.g., jumping, AI), create a new component and a system that processes entities with that component.
- No need to modify existing entity or system classes—just add or remove components as needed.
- Behaviors are composable: an entity can have any combination of components.